A Transformer Based Pipeline for Software Requirements Classification

Maheen Mashrur Hoque
Islamic University of Technology
Gazipur, Bangladesh
maheenmashrur@iut-dhaka.edu

Arman Hossain Dipu
Islamic University of Technology
Gazipur, Bangladesh
armanhossain5@iut-dhaka.edu

Ahsan Habib
Islamic University of Technology
Gazipur, Bangladesh
ahsanhabib3@iut-dhaka.edu

Ajwad Abrar
Islamic University of Technology
Gazipur, Bangladesh
ajwadabrar@iut-dhaka.edu

Abstract

Software Requirements Specification (SRS) documents often contain substantial portions of irrelevant or non-actionable text that obscure true requirements and hinder downstream analysis activities within the Software Development Life Cycle (SDLC). To address this challenge, this paper presents a cost-efficient transformer-based pipeline for distinguishing actionable requirements from noisy sentences. The proposed approach introduces an ensemble pooling strategy that leverages the contextual embeddings of lightweight pretrained transformers, specifically DistilBERT and RoBERTa, and pairs them with ensemble classifiers such as Random Forest, XGBoost, and LightGBM. Using the RegExpPURE dataset, the method achieves improved performance over standard transformer sequence classification heads and outperforms the BERT-based baseline from prior work. The best configuration, DistilBERT paired with Random Forest, attains an accuracy of 78 percent. These results demonstrate that combining compact transformer models with robust ensemble methods provides an effective and resource-friendly solution for requirement noise reduction. The findings further lay the groundwork for extending automated requirement analysis to broader tasks such as requirement assessment, conflict detection, and multimodal requirement understanding.

CCS Concepts

• **Software and its engineering → Requirements analysis**.

Keywords

Software Requirements, Software Development Lifecycle (SDLC), Classification, Natural Language Processing (NLP), Transformers, Ensemble

ACM Reference Format:

Maheen Mashrur Hoque, Arman Hossain Dipu, Ahsan Habib, and Ajwad Abrar. 2018. A Transformer Based Pipeline for Software Requirements Classification. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation emai (Conference acronym 'XX)*. ACM, New York, NY, USA, 6 pages. https://doi.org/XXXXXXX.XXXXXXX

Conference acronym 'XX, Woodstock, NY

ACM ISBN 978-1-4503-XXXX-X/18/06
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Software Requirements and Specifications engineering and collection is a crucial step in the software development lifecycle. Software requirements provide high-level descriptions of a system's expected functionality and constraints, serving as the primary medium through which stakeholder intent is communicated to development teams [13]. To optimize a software development process, it is also important to optimize the requirements and specifications engineering step. So, the domain for this proposed research would be concerned about the optimization of the requirements and specification collection and engineering step.

To be effective, software requirements should maintain clarity, consistency, and completeness [20]. However, real-world Software Requirements Specification (SRS) documents often fall short of this expectation due to communication gaps, inconsistent documentation practices, and varied stakeholder perspectives [12]. These issues result in ambiguous or overly verbose specifications that contain substantial portions of irrelevant or non-actionable information, hereafter referred to as "noise".

The standard industry practice in this regard is to have the system analyst or systems engineer manually validate, organize and classify the client requirement statements and organize them into a technical specification that can be used by the developers and designers later on. Such noise complicates the analytical processes (such as classification, prioritization, and traceability), increasing manual effort for analysts and creating delays within the Software Development Life Cycle (SDLC).

Addressing this challenge requires the ability to automatically distinguish actionable requirements from noisy text. We model this as a Natural Language Processing (NLP) classification task, where a system must determine whether a given sentence constitutes a requirement. In addition to accuracy, cost efficiency is an essential design objective since many organizations and researchers operate with limited computational resources. Motivated by these considerations, we explore the use of lightweight transformer architectures as an effective and scalable solution for requirement noise reduction.

Building on the baseline established by Ivanov et al. [7], we propose an ensemble-based pooling pipeline that leverages contextual embeddings extracted from compact transformer models. The approach integrates these embeddings with ensemble classifiers to improve prediction performance while remaining computationally efficient. The key contributions of this paper are as follows:

- A cost-efficient transformer-based pipeline that uses DistilBERT and RoBERTa with ensemble pooling for requirement classification.
- A novel ensemble pooling strategy that combines transformer embeddings with robust tree-based and boosting-based classifiers.
- An empirical comparison against prior work using the RegExpPURE dataset, demonstrating consistent improvements over BERT-based baselines and traditional shallow models.

2 Related Work

This section reviews prior studies relevant to requirement classification and noise reduction. To provide a broad perspective, we include both traditional rule-based techniques and transformer-based approaches.

2.1 Traditional and Rule-based Approaches

Siahaan and Darnoto introduced the MultiPhiLDA method for detecting irrelevant requirements by modeling topic and word distributions of actor and action terms using polynomial probability functions [16]. Alharbi et al. leveraged semantic embeddings to identify ambiguous requirements, where the embeddings were fine-tuned to capture context-specific indicators of ambiguity [1]. Malik et al. focused on detecting conflicting requirements with cosine similarity over sentence embeddings, which, although aimed at conflicts, provides insight into similarity-based classification techniques [10]. Norabid et al. proposed a rule-based linguistic method that defines extraction rules for dependency relations and part-of-speech patterns, enabling the construction of subject-relation-object triples for downstream analysis [11].

2.2 Transformer-based Approaches

Ivanov et al. investigated requirement extraction using the BERT model [4] fine-tuned on an annotated subset of the PURE corpus [6]. Their model achieved an accuracy of 74 percent, outperforming fastText, which obtained 60 percent accuracy, and ELMo with an SVM classifier, which achieved 59 percent [7]. Since their task and dataset align closely with the objective of distinguishing requirement sentences from non-requirement ones, their work serves as the primary baseline for our study.

3 Dataset

This study uses the PURE (Public Requirements) dataset [6]. It contains 79 Software Requirements Specification (SRS) documents with a total of 34,286 sentences. The dataset provides only raw text. It does not include labels for identifying requirement sentences. Several earlier works, such as [7] and [16], created their own labels to support NLP tasks.

For our experiments, we use the RegExpPURE subset introduced by Ivanov et al. [7]. This subset contains sentence-level labels and has a balanced distribution. The training set has 2474 non-requirements and 2832 requirements. The test set has 467 non-requirements and 1058 requirements. The validation set has 255 non-requirements and 650 requirements. We use the same splits as the authors to ensure a fair comparison with their baseline model.

4 Methodology

Based on the approaches discussed in section 2, we adopt transformer models for the requirement classification task. Our method focuses on a uni-modal setup that uses only text data. This choice is practical because requirement documents are typically provided in textual form, and most information needed for classification is already present in the written descriptions. A multimodal extension, similar to the direction explored by Norabid et al. [11], may be useful in future work, but it is outside the scope of the current study.

4.1 Selected Transformer Models

We use two lightweight transformer models for this study, DistilBERT and RoBERTa. Both models belong to the transformer family, which relies on the self-attention mechanism to model relationships between tokens in a sequence. This mechanism captures long-range dependencies more reliably than recurrent architectures such as RNNs or LSTMs, which often struggle with vanishing gradients and sequential processing constraints [19]. Due to this capability, transformer models have become a standard choice for classification tasks involving natural language.

DistilBERT is a compact version of the original BERT model. It is produced through a student-teacher distillation process that reduces the number of layers while preserving much of the semantic knowledge present in the teacher network [14]. The model matches the teacher's behavior by approximating its logits, hidden states, and attention patterns during training. DistilBERT retains a substantial portion of BERT's accuracy on several NLP benchmarks while requiring significantly fewer parameters. Its reduced memory footprint and computational cost make it suitable for scenarios where efficiency is an important constraint. This motivates its use in our work, as our goal is to design a method that remains accessible without requiring high-end hardware.

RoBERTa is an optimized variant of BERT that builds upon the same architecture but introduces several improvements intended to strengthen the model's language representations [9]. It is trained on a larger and more diverse text corpus and uses dynamic masking during pre-training, which changes the masked positions across epochs. This exposes the model to a wider range of contexts and reduces overfitting to fixed masked patterns. RoBERTa also removes the Next Sentence Prediction task and adjusts key hyperparameters such as batch size and training duration. These changes collectively improve stability during training and boost downstream performance. RoBERTa consistently achieves stronger results than BERT on many NLP tasks, which makes it a strong yet still relatively efficient candidate for requirement classification. For these reasons, we include it alongside DistilBERT in our experiments.

4.2 Definition of Noise in Requirements

In the context of software requirements, *noise* refers to information that is irrelevant, extraneous, or not directly tied to the intended functionality or goals of the system. Such noise may appear in the form of redundant descriptions, historical background, policy statements, or any narrative content that does not express a verifiable requirement. These elements can obscure the true intent of stakeholders and hinder downstream tasks such as requirement classification and analysis.

To illustrate, consider the following examples from the RegExpPURE dataset [7].

Example of a Requirement

Requirement:
System Initialization shall [SRS014] initiate the watchdog timer.

Example of Noise

Noise:
A Working Group was established in May, 2006 to develop high-level functional specifications for an NLM Digital Repository for NLM collection materials and to identify policy and management issues related to the creation, design and management of the repository.

Identifying whether a sentence expresses a requirement often requires interpreting the entire statement and understanding how its terms relate to the behavior of the system. This mirrors the notion of *long-range dependency* in transformer-based models, where the model must capture semantic relationships across a sequence to infer meaning correctly.

4.3 Ensemble Pooling Approach

For the task of noise filtering in software requirements, we introduce a method referred to as *Ensemble Pooling*. The approach is composed of three layers:

(1) **Tokenizer Layer:** This layer converts the input text into its corresponding vectorized token representation.
(2) **Transformer Layer:** The transformer processes the token embeddings to produce contextualized representations of the input sequence.
(3) **Classifier Layer:** Using pooled representations from the transformer outputs, this layer performs the final classification between requirement and noise. During experimentation, ensemble-based classifiers provided superior performance, which motivates the naming of our method as "Ensemble Pooling".

Figure 1 illustrates the overall architecture of the proposed method.

Here, we utilize the final hidden layer of the transformer models, which contains the integrated information or the final contextualized embeddings for the entire input sequence. In the HuggingFace Transformers library [14], this layer is represented with the dimensionality ($Seq_{size} \times Seq_{length} \times Size_{hidden}$), where the default hidden size is 768. Among all the vectors in this layer, the most crucial component is the representation of the special "[CLS]" token. This token is added by the tokenizer to each sequence, and its corresponding embedding acts as an aggregate representation of the entire input [4] [15]. In our method, this "[CLS]" embedding is used as input to the ensemble classifiers for performing the requirement and noise classification.

The DistilBertForSequenceClassification and RobertaForSequenceClassification models (based on DistilBERT and RoBERTa) provided by the Transformers library follow a similar mechanism, but instead place a linear classification head on top of the transformer outputs. These models are included as baselines for comparison in our experiments.

It is also important to note that the transformer layer in our approach does not undergo any training. Since BERT and RoBERTa models have already been pretrained on large English corpora, they inherently possess strong linguistic structure and vocabulary understanding [14] [9]. Leveraging these pretrained models eliminates the need for additional training at the transformer level, reducing both computational cost and training time.

4.4 Classification Layer

The classification layer was trained using the pooled vectors obtained from the transformer layer along with the corresponding labels from the dataset. The labels were mapped to binary values, where 0 denotes noise (not a requirement) and 1 denotes a valid requirement. Four classifier models were trained and evaluated: Random Forest, XGBoost, LightGBM, and SVM. Among these, three are ensemble models (Random Forest, XGBoost, and LightGBM). Ensemble methods combine the decisions of multiple sub-classifiers to achieve improved predictive performance compared to individual models. They also offer increased robustness to overfitting due to their aggregated decision processes [2].

In contrast, SVM is not an ensemble method. It is a standalone classifier that seeks to determine the optimal hyperplane that best separates the data points into distinct classes [3]. We included SVM in our experiments because it has demonstrated strong performance in various NLP classification tasks in prior studies [5] [18] [8].

4.5 Training Parameters

For the experiments, the core DistilBERT and RoBERTa models used for the ensemble pooling approach were not trained specifically for this study. However, the two transformer-based classifier models and the standalone classifiers were trained. The transformer-based classifiers were fine tuned for the requirement classification task using identical hyperparameters for both models. The batch size was set to 16, the maximum sequence length to 128, and the learning rate to 2×10^{-5}. Training was conducted for 10 epochs using the

Table 1: Random Forest Hyperparameters for RoBERTa and DistilBERT Pairings

Hyperparameter	*RoBERTa*	*DistilBERT*
Max Depth (`max_depth`)	None	10
Min Samples Leaf (`min_samples_leaf`)	4	4
Min Samples Split (`min_samples_split`)	2	2
Number of Estimators (`n_estimators`)	300	100

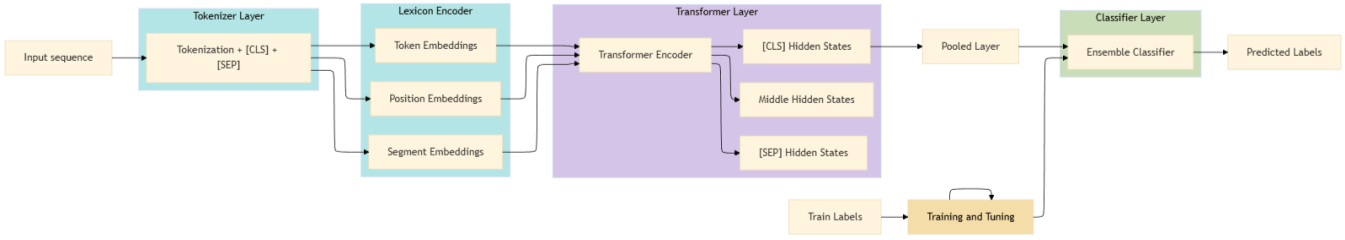

Figure 1: The Ensemble Pooling Classification Approach

Table 2: LightGBM Hyperparameters for RoBERTa and DistilBERT Pairings

Hyperparameter	RoBERTa	DistilBERT
Colsample by Tree (`colsample_bytree`)	1.0	1.0
Learning Rate (`learning_rate`)	0.1	0.1
Min Child Samples (`min_child_samples`)	20	50
Number of Estimators (`n_estimators`)	300	200
Num Leaves (`num_leaves`)	50	50
Subsample (`subsample`)	0.8	0.8

Table 3: XGBoost Hyperparameters for RoBERTa and DistilBERT Pairings

Hyperparameter	RoBERTa	DistilBERT
Learning Rate (`learning_rate`)	0.1	0.1
Max Depth (`max_depth`)	5	5
Number of Estimators (`n_estimators`)	300	300
Subsample (`subsample`)	0.8	0.7
Colsample by Tree (`colsample_bytree`)	0.8	1.0
Gamma (`gamma`)	0.1	0

Adam optimizer, with Cross Entropy Loss as the loss function. Although both DistilBERT and RoBERTa are built upon the BERT architecture and share the same parameter count, they differ in how they internally process sequences (as discussed in section 4).

For the non-transformer classifiers, all models were fine tuned using a 10 fold grid search cross validation setup. Accuracy was chosen as the target metric for hyperparameter optimization. The optimal hyperparameters for each model are presented in Table 1, Table 2, Table 3 and Table 4. Here, the term "Pairing" refers to the process of feeding the hidden layer representations of the language models into the classifier models to generate predictions.

5 Results

Following the setup described in section 4, we compared our experimental outcomes with those reported in the baseline study. As in the baseline work, all results are computed on the validation set. For comparison, we primarily relied on the accuracy metric, and the justification for this choice is discussed in section 6. The results

Table 4: SVM Hyperparameters for RoBERTa and DistilBERT Pairings

Hyperparameter	RoBERTa	DistilBERT
C (`C`)	10	10
Gamma (`gamma`)	scale	auto
Kernel (`kernel`)	rbf	rbf

of our experiments are presented in Table 5, alongside the baseline for comparison.

For our experiments and the proposed approach to identifying requirement noise, we used the RegExpPURE dataset [7]. Consequently, our baseline consists of the results reported in the same paper, where the authors used the BERT base model to extract requirements. Although their work does not explicitly frame the task as noise reduction, their objective aligns closely with ours. As shown in section 3, the dataset includes labeled requirement and non-requirement sentences. These non-requirement sentences correspond directly to what we define as noise. Therefore, using their work as a baseline is both appropriate and well justified.

Table 5: Comparison of Our Approaches with Baseline Results

Model / Technique	Category	Accuracy (%)
fastText (Baseline)	Baseline	60
ELMo + SVM (Baseline)	Baseline	59
BERT (Baseline)	Baseline	74
DistilBERT Sequence Classifier	DistilBERT	76
DistilBERT + Random Forest	DistilBERT	**78**
DistilBERT + XGB	DistilBERT	77
DistilBERT + LightGBM	DistilBERT	77
DistilBERT + SVM	DistilBERT	76
RoBERTa Sequence Classifier	RoBERTa	77
RoBERTa + Random Forest	RoBERTa	75
RoBERTa + XGB	RoBERTa	77
RoBERTa + LightGBM	RoBERTa	77
RoBERTa + SVM	RoBERTa	77

6 Discussion

6.1 The Rationale for Accuracy

Accuracy is an intuitive and widely understood evaluation metric, as it directly reflects the proportion of correct predictions made by the model. In many classification scenarios, achieving balanced performance across classes is desirable, and accuracy provides an equal weighting of all classes. Since the class distribution in the RegExpPURE dataset is approximately uniform, accuracy serves as an appropriate and reliable metric for assessing model performance. For these reasons, and consistent with prior recommendations in the literature [17], accuracy was chosen as the primary evaluation metric in this study.

6.2 Base Classifier vs Ensemble Pooling

The results in Table 5 show that our approach, which combines DistilBERT with a Random Forest classifier in the ensemble pooling setup, achieves higher accuracy than all other tested methods as well as the baseline results. The approaches involving RoBERTa produced scores that were relatively close to one another, with the exception of Random Forest. This deviation is most likely influenced by how RoBERTa applies byte pair encoding during tokenization and how its encoder layers adapt to contextual information in the sequence [9].

The baseline paper reports its strongest performance using BERT, reaching approximately 74% accuracy. In that baseline, the BERT classifier, similar to DistilBERT and RoBERTa classifiers, employs a neural network layer with a linear softmax activation function to produce class probabilities. While such architectures are effective for many tasks, our classification setting benefits from more robust methods such as ensemble classifiers. Ensemble methods aggregate the outputs of multiple sub-classifiers and often deliver improved predictive performance. Random Forest is one such method that combines decision trees, where each tree contributes to the final decision. This aggregated decision-making improves robustness and accuracy, and this effect is reflected in our experimental results. DistilBERT and RoBERTa were chosen due to their lightweight nature. They require fewer computational resources than larger transformer models and remain practical options for smaller NLP tasks.

However, the lightweight nature of these models also poses a limitation. The accuracy values remain below the 80 percent range, which suggests that these models are constrained by their relatively small parameter counts when compared with modern large-scale models such as GPT, Gemini, or LLaMA. This observation is aligned with the insights from the Densing Law of LLMs [21], which relates model capacity to performance. As a future research direction for requirement noise reduction, exploring the use of state-of-the-art large language models and more sophisticated classification strategies, such as stacking multiple classifiers, represents a promising and yet unexplored opportunity.

7 Conclusion

This work investigated the use of transformer representations combined with ensemble classifiers to identify noise in software requirements. Through the proposed ensemble pooling approach, we demonstrated that lightweight pretrained models such as DistilBERT and RoBERTa, when paired with ensemble methods like Random Forest, can outperform standard transformer classification heads and the baseline reported in prior work. The results suggest that robust downstream classifiers can effectively complement pretrained language model embeddings in requirement classification tasks while maintaining low computational cost.

Although the study focused on noise reduction, the findings provide a foundation for extending automated requirement analysis pipelines. Future work includes incorporating advanced large language models, exploring stacked ensemble architectures, and expanding the scope beyond noise detection to address requirement assessment, conflict identification, and multimodal analysis involving diagrams and related artifacts. These directions present promising opportunities to further support practitioners in improving the clarity and quality of software requirements.

References

[1] Sadeen Alharbi. 2022. Ambiguity Detection in Requirements Classification Task using Fine-Tuned Transformation Technique. In *CS & IT Conference Proceedings*, Vol. 12. CS & IT Conference Proceedings.

[2] Leo Breiman. 2001. Random forests. *Machine learning* 45 (2001), 5–32.

[3] Corinna Cortes and Vladimir Vapnik. 1995. Support-vector networks. *Machine learning* 20 (1995), 273–297.

[4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).

[5] Harris Drucker, Donghui Wu, and Vladimir N Vapnik. 1999. Support vector machines for spam categorization. *IEEE Transactions on Neural networks* 10, 5 (1999), 1048–1054.

[6] Alessio Ferrari, Giorgio Oronzo Spagnolo, and Stefania Gnesi. 2017. Pure: A dataset of public requirements documents. In *2017 IEEE 25th International Requirements Engineering Conference (RE)*. IEEE, 502–505.

[7] Vladimir Ivanov, Andrey Sadovykh, Alexandr Naumchev, Alessandra Bagnato, and Kirill Yakovlev. 2021. Extracting software requirements from unstructured documents. In *International Conference on Analysis of Images, Social Networks and Texts*. Springer, 17–29.

[8] Thorsten Joachims. 1998. Text categorization with support vector machines: Learning with many relevant features. In *European conference on machine learning*. Springer, 137–142.

[9] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* (2019).

[10] Garima Malik, Mucahit Cevik, Devang Parikh, and Ayse Basar. 2022. Identifying the requirement conflicts in SRS documents using transformer-based sentence embeddings. *arXiv preprint arXiv:2206.13690* (2022).

[11] Idza Aisara Norabid and Fariza Fauzi. 2022. Rule-based text extraction for multimodal Knowledge Graph. *International Journal of Advanced Computer Science and Applications* 13, 5 (2022).

[12] Bashar Nuseibeh and Steve Easterbrook. 2000. Requirements engineering: a roadmap. In *Proceedings of the Conference on the Future of Software Engineering*. 35–46.

[13] Klaus Pohl. 2016. *Requirements engineering fundamentals: a study guide for the certified professional for requirements engineering exam-foundation level-IREB compliant*. Rocky Nook, Inc.

[14] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108* (2019).

[15] Justyna Sarzynska-Wawer, Aleksander Wawer, Aleksandra Pawlak, Julia Szymanowska, Izabela Stefaniak, Michal Jarkiewicz, and Lukasz Okruszek. 2021. Detecting formal thought disorder by deep contextualized word representations. *Psychiatry Research* 304 (2021), 114135.

[16] Daniel Siahaan and Brian Rizqi Paradisiaca Darnoto. 2022. A Novel Framework to Detect Irrelevant Software Requirements Based on MultiPhiLDA as the Topic Model. In *Informatics*, Vol. 9. MDPI, 87.

[17] Marina Sokolova and Guy Lapalme. 2009. A systematic analysis of performance measures for classification tasks. *Information processing & management* 45, 4 (2009), 427–437.

[18] Kentaro Torisawa et al. 2007. A new perceptron algorithm for sequence labeling with non-local features. In *Proceedings of the 2007 Joint Conference on Empirical*

Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL). 315–324.

[19] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[20] Karl E Wiegers and Joy Beatty. 2013. *Software requirements*. Pearson Education.

[21] Chaojun Xiao, Jie Cai, Weilin Zhao, Biyuan Lin, Guoyang Zeng, Jie Zhou, Zhi Zheng, Xu Han, Zhiyuan Liu, and Maosong Sun. 2025. Densing law of llms. *Nature Machine Intelligence* (2025), 1–11.